Teste de Software

# Detecção de Bad Smells e Refatoração Segura

Relatório do trabalho prático — ESLint + eslint-plugin-sonarjs + Jest

**Aluno:** Arthur Candian
**Matrícula:** 804243

# 1. Análise manual de Bad Smells

O arquivo analisado foi `src/ReportGenerator.js`, cujo único método, `generateReport`, tem 58 linhas e concentra cabeçalho, regras de acesso por perfil, formatação por tipo de relatório, soma do total e rodapé. Identifiquei três smells principais:

### 1.1 Método Longo (Long Method) com condicionais aninhadas

Um único método faz cinco coisas e chega a quatro níveis de aninhamento (*for → if role → if valor → if tipo*). É problemático para a manutenção porque, para alterar qualquer regra, é preciso ler e entender o método inteiro; qualquer mudança pode afetar um ramo que não tinha relação com ela. Para os testes, a combinação tipo × perfil × valor fica toda dentro de uma só função: não é possível testar a formatação sem passar pela regra de acesso, nem o contrário, e cada novo tipo de relatório ou perfil multiplica os caminhos a cobrir.

### 1.2 Código Duplicado (Duplicated Code)

A linha de item em CSV está escrita duas vezes (ramos ADMIN e USER), a linha de item em HTML também, o `total += item.value` aparece em quatro ramos e o teste `reportType === 'CSV' / 'HTML'` se repete no cabeçalho, no corpo (duas vezes) e no rodapé. Alterar o formato de uma coluna exige editar vários pontos (*shotgun surgery*), e esquecer um deles gera inconsistência. Essa inconsistência de fato existe: a cópia HTML do ramo ADMIN escreve `<tr ${style}>` e produz `<tr >` (com espaço) quando o item não é prioritário, enquanto a cópia do ramo USER escreve `<tr>`. Foi exatamente esse defeito que o teste do Admin acusou (seção 2).

### 1.3 Números Mágicos (Magic Numbers) e strings soltas

Os valores `1000` (limite de prioridade do ADMIN) e `500` (limite de visibilidade do USER) aparecem diretamente nas condições, e o significado de negócio só está em comentários. Além disso, as strings `'ADMIN'`, `'USER'`, `'CSV'` e `'HTML'` estão espalhadas pelo código. Quem lê não sabe se um 500 é o mesmo 500 de outra regra, mudar um limite exige caçar ocorrências, e os testes ficam acoplados a números sem nome. Observei ainda um efeito colateral oculto (o método altera `item.priority` nos objetos recebidos), que preservei na refatoração para não mudar o comportamento.

# 2. Relatório da ferramenta (ESLint + SonarJS)

Com as regras `plugin:sonarjs/recommended` e o limite de complexidade cognitiva já configurado no `.eslintrc.json` do projeto (limite 5), executei a análise no código original, antes de qualquer alteração:

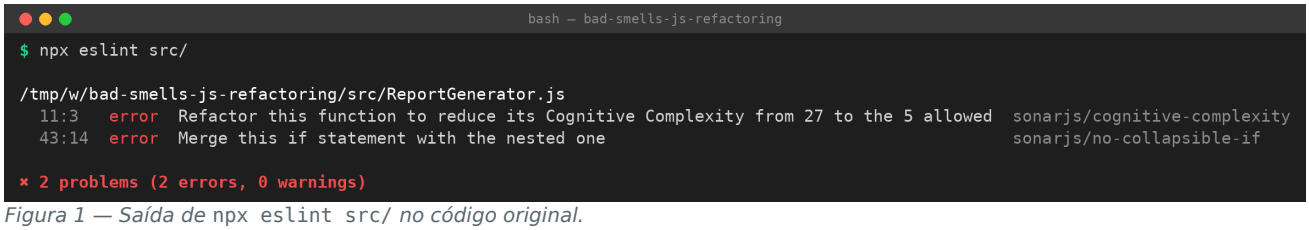



*Figura 1 — Saída de `npx eslint src/` no código original.*

A ferramenta apontou 2 erros. O **sonarjs/cognitive-complexity** mediu a complexidade cognitiva de `generateReport` em **27** (limite 5; mesmo o limite padrão de 15 seria excedido). Esse número ajuda onde a análise manual é fraca: eu percebi que o método era "grande e confuso", mas a métrica quantifica o problema de forma objetiva, penalizando cada nível de aninhamento, o que o olho humano tende a subestimar, e permite verificar depois se a refatoração realmente melhorou. O **sonarjs/no-collapsible-if** (linha 43) apontou um micro-smell que eu não tinha anotado: o `if (item.value <= 500)` é o único conteúdo do `else if` do perfil USER e poderia ser fundido à condição externa.

Em contrapartida, a ferramenta **não** reportou a duplicação de código, os números mágicos nem o efeito colateral em `item.priority`; esses três só apareceram na leitura manual. Também não detectou o bug do `<tr >`, que foi revelado pela suíte de testes. As duas abordagens se complementam: análise estática para métricas e padrões objetivos, leitura humana para intenção e design, e testes para comportamento.

### Rede de segurança no estado inicial

Após corrigir o script de teste no `package.json` (o projeto usa ES Modules e o Jest precisa de `--experimental-vm-modules`), a suíte original apresentou 4 testes passando e 1 falhando, justamente o do HTML do Admin, por causa do `<tr >` descrito acima:

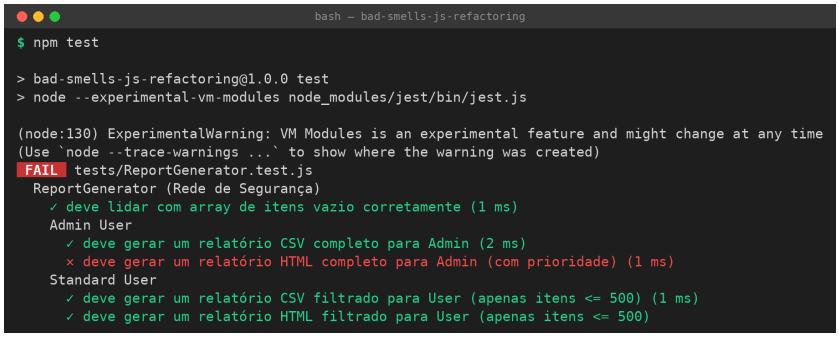



*Figura 2 — Recorte da saída de* `npm test` *no código original.*

# 3. Processo de refatoração

O smell mais crítico era o **Método Longo com complexidade cognitiva 27**, pois ele era a raiz dos demais (duplicação e condicionais repetidas). A refatoração foi feita em `src/ReportGenerator.refactored.js`, sem alterar o original, e validada por uma cópia da suíte apontando para o novo arquivo.

### Antes — laço principal do método original (linhas 27–55)

```
    for (const item of items) {
      if (user.role === 'ADMIN') {
        // Admins veem todos os itens
        if (item.value > 1000) {
          // Lógica bônus para admins
          item.priority = true;
        }

        if (reportType === 'CSV') {
          report += `${item.id},${item.name},${item.value},${user.name}\n`;
          total += item.value;
        } else if (reportType === 'HTML') {
          const style = item.priority ? 'style="font-weight:bold;"' : '';
          report += `<tr ${style}><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
          total += item.value;
        }
      } else if (user.role === 'USER') {
        // Users comuns só veem itens de valor baixo
        if (item.value <= 500) {
          if (reportType === 'CSV') {
            report += `${item.id},${item.name},${item.value},${user.name}\n`;
            total += item.value;
          } else if (reportType === 'HTML') {
            report += `<tr><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
            total += item.value;
          }
        }
      }
    }
```

### Depois — `renderBody` e `generateReport` (formatadores e políticas ficam em objetos separados)

```
function renderBody(items, user, formatter, policy) {
  let body = '';
  let total = 0;

  for (const item of items) {
    policy.prepare(item);
    if (!policy.isVisible(item)) {
      continue;
    }
    body += formatter.row(item, user, policy.isHighlighted(item));
    total += item.value;
  }

  return { body, total };
}

  generateReport(reportType, user, items) {
    const formatter = FORMATTERS.get(reportType) ?? nullFormatter;
    const policy = POLICIES.get(user?.role) ?? noAccessPolicy;

    const { body, total } = renderBody(items, user, formatter, policy);

    return (formatter.header(user) + body + formatter.footer(total)).trim();
  }
```

### Técnicas aplicadas

- **Extract Function / Extract Method:** o corpo do laço virou `renderBody`; cabeçalho, linha e rodapé de cada formato viraram funções pequenas dos formatadores. O `generateReport` passou de 58 para 8 linhas e agora só coordena.
- **Replace Conditional with Polymorphism:** os `if/else` por tipo (CSV/HTML) e por perfil (ADMIN/USER) foram trocados por objetos intercambiáveis com a mesma interface (`header/row/footer` e `prepare/isVisible/isHighlighted`), selecionados por um `Map`. Adicionar um novo tipo ou perfil passa a ser incluir um objeto, sem mexer no fluxo.
- **Eliminação de duplicação:** cada formato de linha existe uma só vez e o `total += item.value` aparece em um único lugar. A duplicação eliminada também corrigiu o `<tr >`.
- **Replace Magic Number with Symbolic Constant:** `ADMIN_PRIORITY_THRESHOLD = 1000` e `USER_MAX_VISIBLE_VALUE = 500`.
- **Introduce Null Object:** `nullFormatter` e `noAccessPolicy` tratam tipo ou perfil desconhecidos sem condicionais, mantendo o comportamento original (relatório vazio).

Resultado medido pelo SonarJS: a complexidade cognitiva máxima por função caiu de **27** para **3** (`renderBody`); `generateReport` ficou com 2 e as demais funções com 0 ou 1.

## 4. Validação final

Executei o ESLint no arquivo refatorado (sem nenhum erro, código de saída 0) e a suíte completa. Para que o arquivo original também passasse, apliquei nele uma correção mínima de uma linha, trocando `<tr ${style}>` por `<tr${style ? " " + style : ""}>`, em um commit separado. Resultado: **2 suítes e 10 testes passando**, o que mostra que a refatoração preservou o comportamento esperado.




*Figura 3 — `npx eslint src/ReportGenerator.refactored.js`: nenhum erro (código de saída 0).*

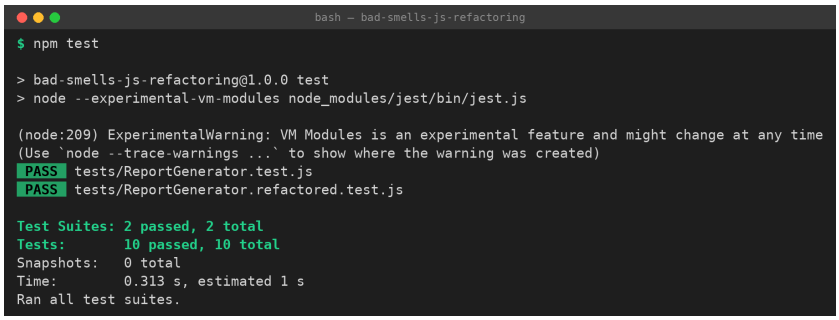



*Figura 4 — `npm test` com o original (corrigido) e o refatorado: 10/10 testes passando.*

## 5. Conclusão

Os testes foram o que tornou a refatoração segura: ao trocar a estrutura inteira do método, eu podia rodar a suíte a cada passo e saber na hora se o comportamento observável tinha mudado. Sem essa rede, reescrever uma função de complexidade 27 seria um risco alto; com ela, a mudança vira uma sequência de passos pequenos e verificáveis. Os testes também revelaram um bug real (`<tr >`) que a análise estática não viu, mostrando que as duas ferramentas cobrem falhas diferentes.

Reduzir bad smells melhora a qualidade geral do software: o código ficou mais legível (cada função tem uma responsabilidade), mais fácil de testar isoladamente (formatadores e políticas podem ser testados separadamente), mais fácil de evoluir (novos formatos ou perfis são adições, não edições em cascata) e menos sujeito a divergências entre cópias de uma mesma regra. A complexidade cognitiva de 27 para no máximo 3 resume esse ganho de forma objetiva.